

Group 4 Graph Theory

Naura Rizky Ameira - 5025251001

Farras Al Ghifari - 5025251006

Nisrina Zahralilla - 5025251013

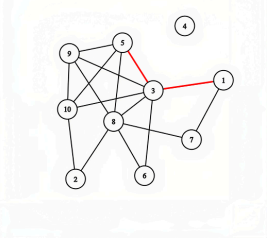
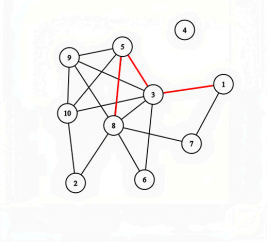
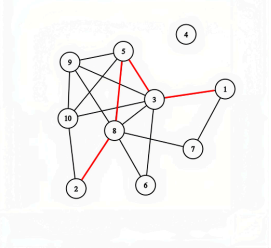
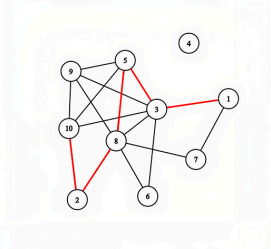
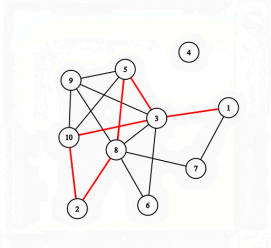
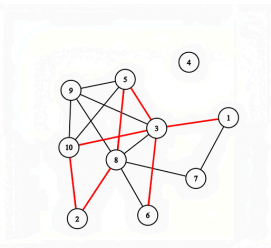
Safiya Nadira Az-zahra - 5025251268

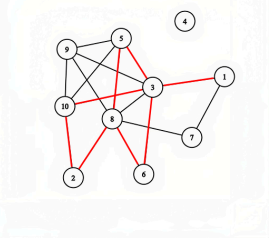
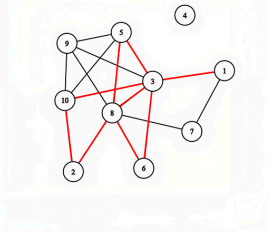
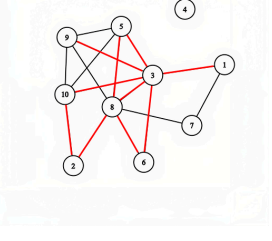
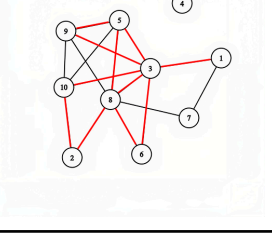
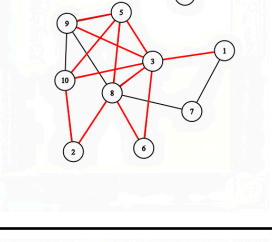
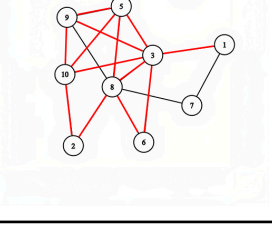
1. Fleury's Algorithm

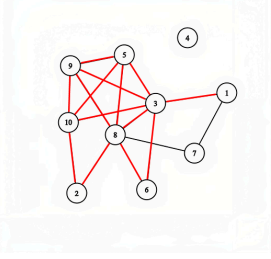
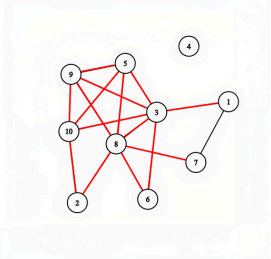
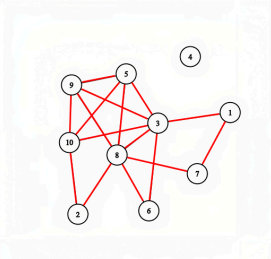
Sample test case:

Input	Output	Graph Visualization
10 16 9 10 8 6 8 5 1 7 2 8 3 8 3 1 6 3 10 3 8 9 3 5 10 2 5 9 7 8 9 3 5 10	1 3 5 8 2 10 3 6 8 3 9 5 10 9 8 7 1	

Step	Current	Candidate Edges	Edge Chosen	Decision	Resulting Vertex	Graph
1	1	1-3, 1-7	1-3	Not a bridge	3	

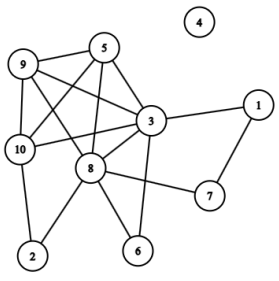
2	3	3-5, 3-6, 3-8, 3-9, 3-10	3-5	Not a bridge	5	
3	5	5-8, 5-9, 5-10	5-8	Not a bridge	8	
4	8	8-2, 8-3, 8-6, 8-7, 8-9	8-2	Not a bridge	2	
5	2	2-10	2-10	Only edge available	10	
6	10	10-3, 10-5, 10-9	10-3	Not a bridge	3	
7	3	3-6, 3-8, 3-9	3-6	Not a bridge	6	

8	6	6-8	6-8	Only edge available	8	
9	8	8-3, 8-7, 8-9	8-3	Not a bridge	3	
10	3	3-9	3-9	Only edge available	9	
11	9	9-5, 9-8, 9-10	9-5	Not a bridge	5	
12	5	5-10	5-10	Only edge available	10	
13	10	10-9	10-9	Only edge available	9	

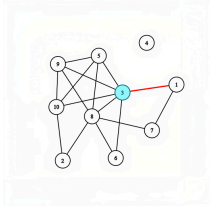
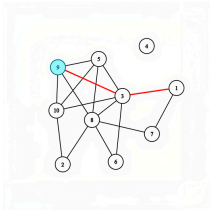
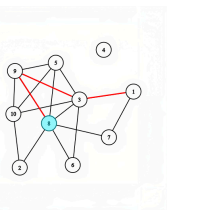
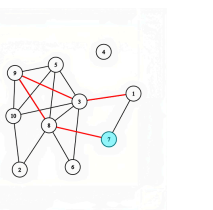
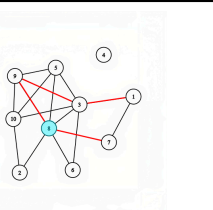
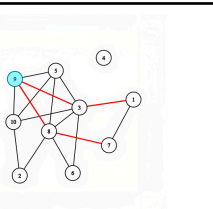
14	9	9-8	9-8	Only edge available	8	
15	8	8-7	8-7	Only edge available	7	
16	7	7-1	7-1	Only edge available	1	

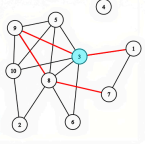
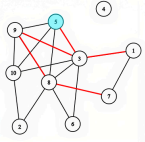
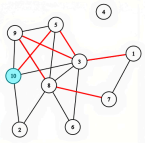
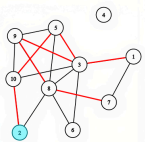
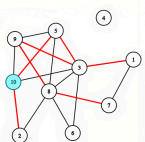
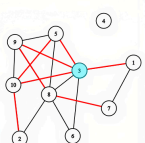
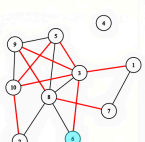
2. Hierholzer's Algorithm

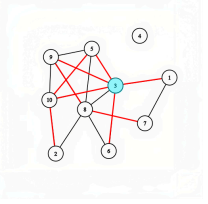
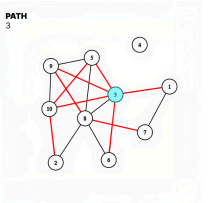
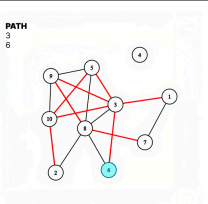
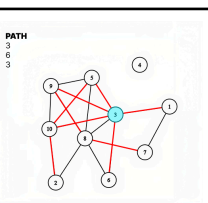
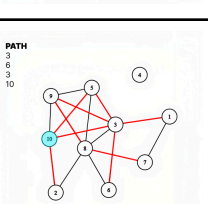
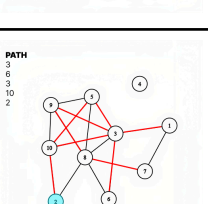
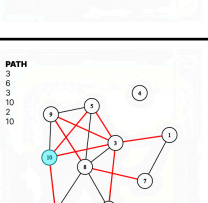
Sample test case:

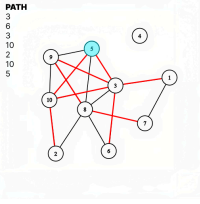
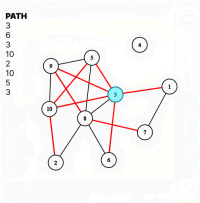
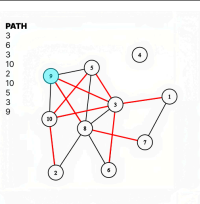
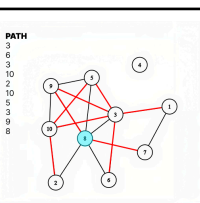
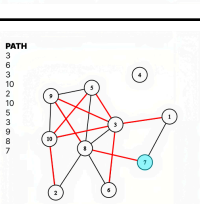
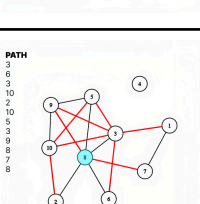
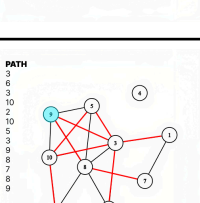
Input	Output	Graph Visualization
10 16 9 10 8 6 8 5 1 7 2 8 3 8 3 1 6 3 10 3 8 9 3 5 10 2 5 9 7 8	1 3 5 8 2 10 3 6 8 3 9 5 10 9 8 7 1	

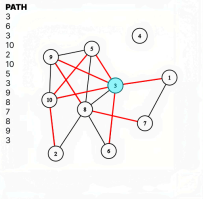
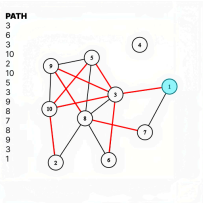
9 3 5 10		
-------------	--	--

Step	Current	Available Edges	Action	Stack / Result	Graph
1	1	1-3	Traverse 1-3	Stack: 1 > 3	
2	3	3-9, 3-5, 3-10, 3-8, 3-6	Traverse 3-9	Stack: 1 > 3 > 9	
3	9	9-8, 9-3, 9-8, 9-5	Traverse 9-8	Stack: 1 > 3 > 9 > 8	
4	8	8-7, 8-9, 8-7, 8-9, 8-3, 8-5	Traverse 8-7	Stack: 1 > 3 > 9 > 8 > 7	
5	7	7-8	Traverse 7-8	Stack: 1 > 3 > 9 > 8 > 7 > 8	
6	8	8-9, 8-7, 8-9, 8-3, 8-5	Traverse 8-9	Stack: 1 > 3 > 9 > 8 > 7 > 8 > 9	

7	9	9-3, 9-8, 9-5	Traverse 9-3	Stack: 1 > 3 > 9 > 8 > 7 > 8 > 9 > 3	
8	3	3-9, 3-5, 3-10, 3-8, 3-6	Traverse 3-5	Stack: 1 > 3 > 9 > 8 > 7 > 8 > 9 > 3 > 5	
9	5	5-10, 5-3, 5-9, 5-8	Traverse 5-10	Stack: ... > 5 > 10	
10	10	10-5, 10-2, 10-3, 10-2	Traverse 10-2	Stack: ... > 10 > 2	
11	2	2-10	Traverse 2-10	Stack: ... > 2 > 10	
12	10	10-3, 10-2	Traverse 10-3	Stack: ... > 10 > 3	
13	3	3-9, 3-10, 3-8, 3-6	Traverse 3-6	Stack: ... > 3 > 6	

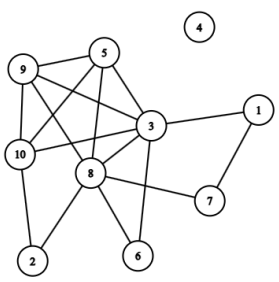
14	6	6-3	Traverse 6-3	Stack: ... > 6 > 3	
15	3	No unused edges	Backtrack / record 3	3 added to path	
16	6	No unused edges	Backtrack / record 6	6 added to path	
17	3	No unused edges	Backtrack / record 3	3 added to path	
18	10	No unused edges	Backtrack / record 10	10 added to path	
19	2	No unused edges	Backtrack / record 2	2 added to path	
20	10	No unused edges	Backtrack / record 10	10 added to path	

21	5	No unused edges	Backtrack / record 5	5 added to path	
22	3	No unused edges	Backtrack / record 3	3 added to path	
23	9	No unused edges	Backtrack / record 9	9 added to path	
24	8	No unused edges	Backtrack / record 8	8 added to path	
25	7	No unused edges	Backtrack / record 7	7 added to path	
26	8	No unused edges	Backtrack / record 8	8 added to path	
27	9	No unused edges	Backtrack / record 9	9 added to path	

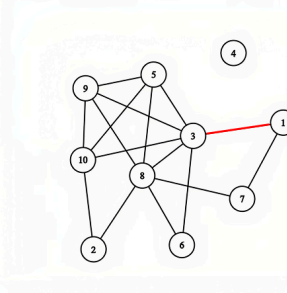
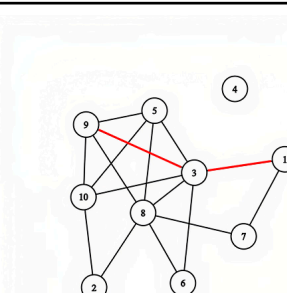
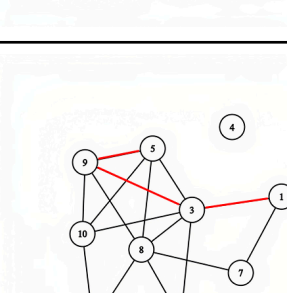
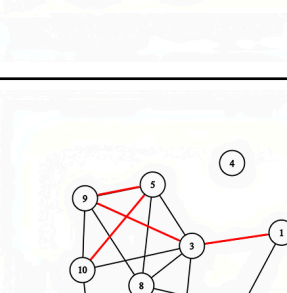
28	3	No unused edges	Backtrack / record 3	3 added to path	
29	1	No unused edges	Backtrack / record 1	1 added to path	

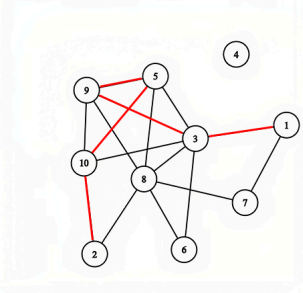
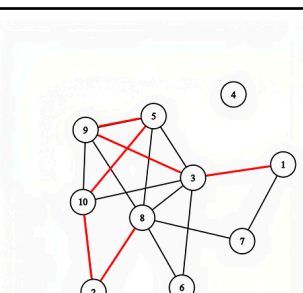
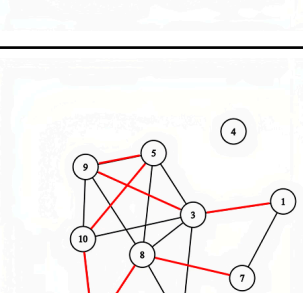
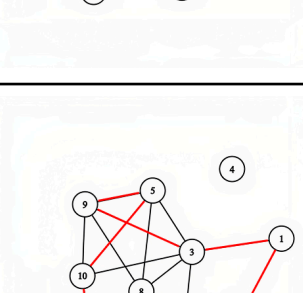
3. Tucker's Algorithm

Sample test case:

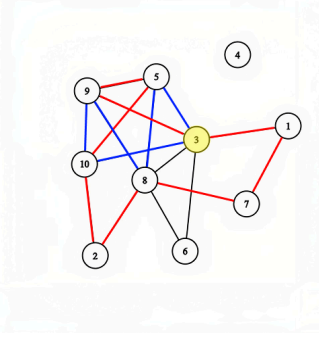
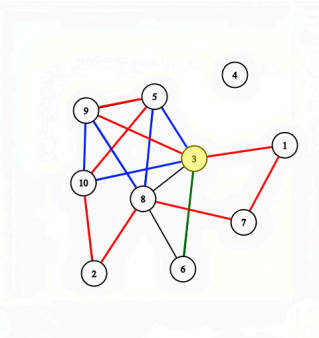
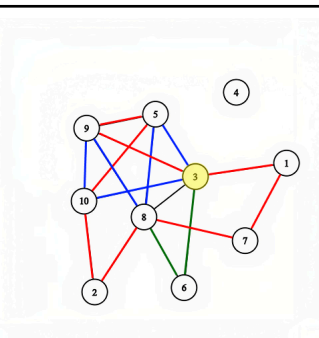
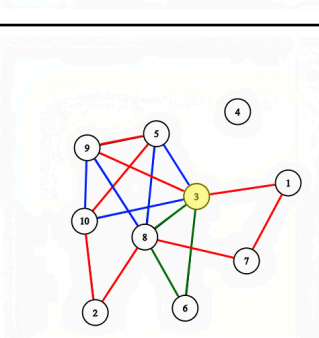
Input	Output	Graph Visualization
10 16 9 10 8 6 8 5 1 7 2 8 3 8 3 1 6 3 10 3 8 9 3 5 10 2 5 9 7 8 9 3 5 10	1 3 5 8 2 10 3 6 8 3 9 5 10 9 8 7 1	

Step	Phase	Current	Available Unused Edges	Action	Tour / Result	Graph
------	-------	---------	---------------------------	--------	------------------	-------

1	Initial Cycle	1	1-3, 1-7	Traverse 1-3	1 > 3	
2	Initial Cycle	3	3-9, 3-5, 3-10, 3-6, 3-8	Traverse 3-9	1 > 3 > 9	
3	Initial Cycle	9	9-5, 9-8, 9-10	Traverse 9-5	1 > 3 > 9 > 5	
4	Initial Cycle	5	5-10, 5-8, 5-3, 5-9	Traverse 5-10	1 > 3 > 9 > 5 > 10	

5	Initial Cycle	10	10-2, 10-3, 10-9	Traverse 10-2	$1 > 3 > 9 > 5$ $> 10 > 2$	
6	Initial Cycle	2	2-8	Traverse 2-8	$1 > 3 > 9 > 5$ $> 10 > 2 > 8$	
7	Initial Cycle	8	8-7, 8-9, 8-3, 8-5, 8-6	Traverse 8-7	$1 > 3 > 9 > 5$ $> 10 > 2 > 8$ > 7	
8	Initial Cycle	7	7-1	Traverse 7-1	Initial cycle completed	

9	Absorption 1	3	3-5, 3-10, 3-6, 3-8	Start new cycle	3	
10	Absorption 1	5	5-8	Traverse 5-8	$3 > 5 > 8$	
11	Absorption 1	8	8-9, 8-3, 8-6	Traverse 8-9	$3 > 5 > 8 > 9$	
12	Absorption 1	9	9-10	Traverse 9-10	$3 > 5 > 8 > 9 > 10$	

13	Absorption 1	10	10-3	Traverse 10-3	Cycle completed	
14	Absorption 2	3	3-6, 3-8	Start new cycle	3	
15	Absorption 2	6	6-8	Traverse 6-8	$3 > 6 > 8$	
16	Absorption 2	8	8-3	Traverse 8-3	Cycle completed	

Analysis Result:

Time Complexity	
Fleury's Algorithm	$O(EV^3)$

Hierholzer's Algorithm	$O(V + E)$
Tucker's Algorithm	$O(V + E)$

Based on the implemented algorithms, Fleury's Algorithm has a significantly higher time complexity than both Tucker's and Hierholzer's Algorithms. Therefore, for this implementation, Hierholzer's or Tucker's Algorithm can achieve the same Eulerian tour with lower computational complexity.

Hierholzer's and Tucker's Algorithms have the same time complexity, $O(V + E)$, but use different approaches to construct the Eulerian tour. Hierholzer's Algorithm uses a stack-based traversal and backtracking approach, while Tucker's Algorithm constructs smaller cycles and progressively absorbs them into the existing tour.

In terms of the number of iterations recorded in this implementation, Tucker's Algorithm requires 16 iterations, whereas Hierholzer's Algorithm requires 29 iterations. Thus, Tucker's Algorithm produces the required Eulerian tour with fewer recorded iterations in this particular input.